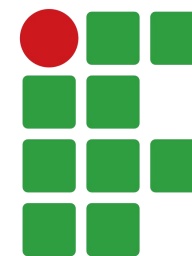


PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite

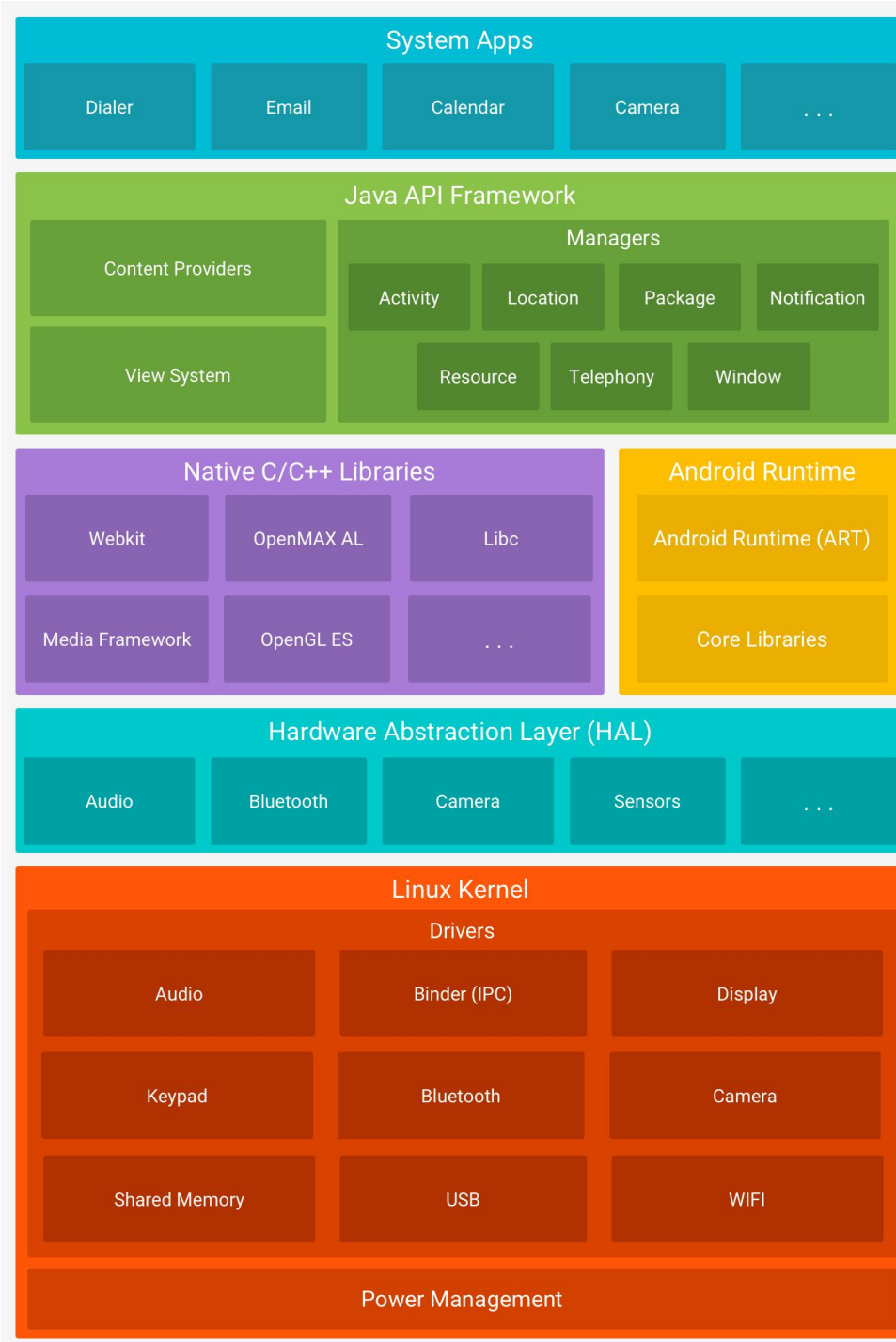


**INSTITUTO
FEDERAL**
Pernambuco

SQLite

- SQLite é um banco de dados relacional open source que oferece suporte à linguagem SQL para criação, consulta e manipulação de dados.
- É uma biblioteca escrita em C que implementa um mecanismo de banco de dados SQL, sendo leve, eficiente e adequado para aplicações que precisam armazenar dados localmente, como aplicativos Android.
- Uma aplicação Android pode criar um ou mais bancos de dados SQLite, que são armazenados no próprio dispositivo. Por padrão, esses bancos são privados à aplicação que os criou, não podendo ser acessados diretamente por outras aplicações.

SQLite



SQLite

- O Android inclui uma implementação do SQLite no próprio sistema operacional.
- Permite utilizar diretamente APIs do Android, como SQLiteDatabase e SQLiteOpenHelper, sem a necessidade de adicionar uma biblioteca externa para utilizar o SQLite.
- O banco é armazenado localmente no dispositivo, dentro do espaço privado da aplicação.
- O Android também oferece o Room, que é uma camada de abstração sobre o SQLite, facilitando o trabalho com banco de dados e reduzindo código repetitivo.

Room

- O Room é uma biblioteca de persistência do Android que fornece uma camada de abstração sobre o SQLite. Ele facilita o trabalho com bancos de dados locais, permitindo que o desenvolvedor utilize objetos Kotlin e interfaces para definir as estruturas e operações do banco.
- Room organiza o acesso aos dados principalmente em três componentes:
 - Entity: representa uma tabela do banco de dados.
 - DAO (Data Access Object): define as operações realizadas sobre os dados, como inserir, consultar, atualizar e excluir registros.
 - Database: representa o banco de dados e fornece acesso aos DAOs.

DAO (Data Access Object)

- O DAO (Data Access Object) é o componente responsável por definir as operações de acesso e manipulação dos dados no banco de dados.
- No Room, o DAO normalmente é declarado como uma interface, na qual são definidas operações como inserir, consultar, atualizar e excluir registros.
- O Room gera automaticamente a implementação do DAO durante a compilação.

```
@Dao
interface UserDao {
    @Insert
    suspend fun insertAll(vararg users: User)

    @Delete
    suspend fun delete(user: User)

    @Query("SELECT * FROM user")
    suspend fun getAll(): List<User>
}
```

DAO (Data Access Object)

- O DAO (Data Access Object) é o componente responsável por definir as operações de acesso e manipulação dos dados no banco de dados.
- No Room, o DAO normalmente é declarado como uma interface, na qual são definidas operações como inserir, consultar, atualizar e excluir registros.
- O Room gera automaticamente a implementação do DAO durante a compilação.

O `suspend` é utilizado para indicar que uma operação de acesso ao banco pode ser executada de forma assíncrona, sem bloquear a execução da aplicação enquanto o banco é acessado.

Fonte: <https://developer.android.com/training/data-storage/room/accessing-data?hl=pt-br>

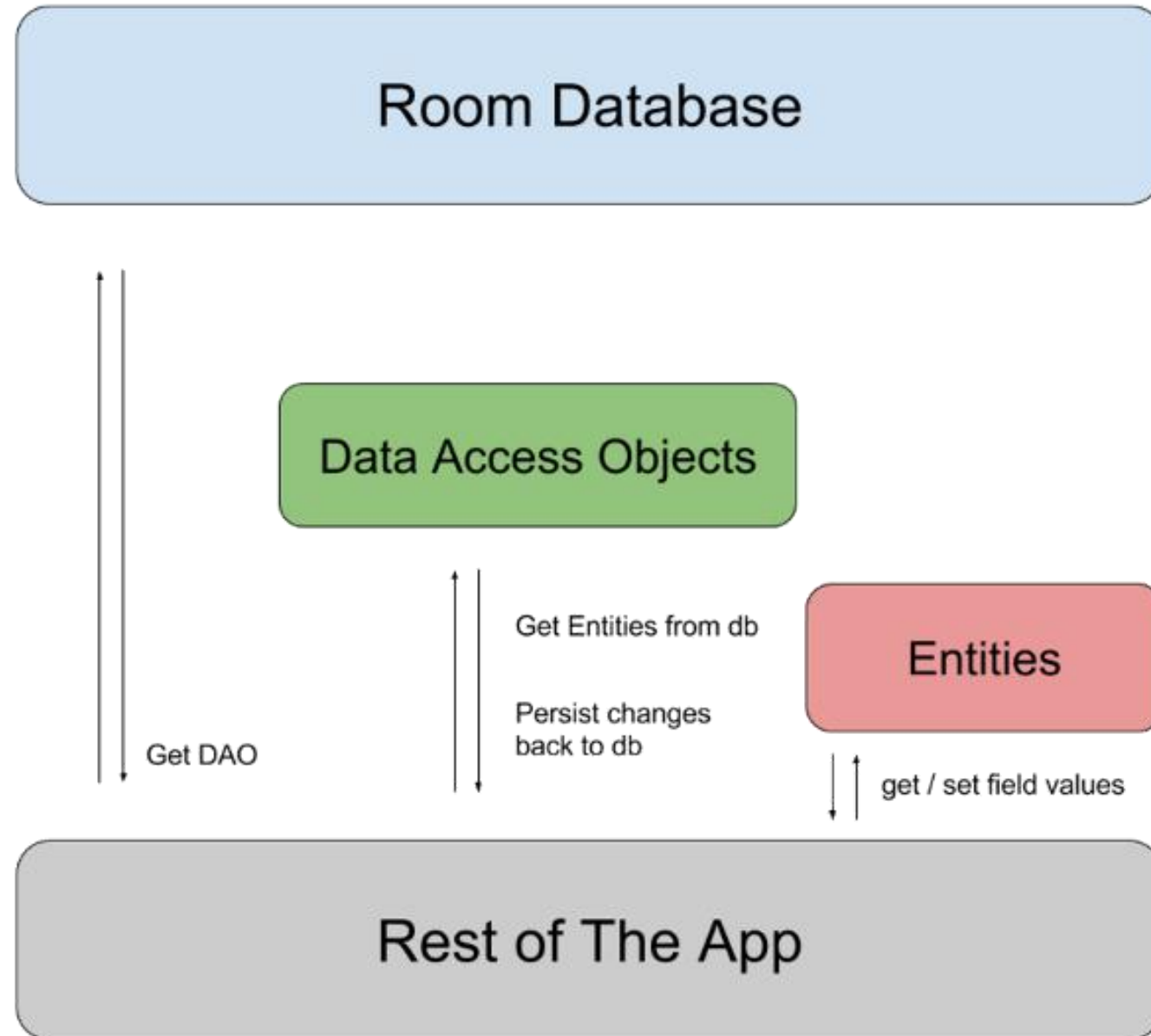
```
@Dao
interface UserDao {
    @Insert
    suspend fun insertAll(vararg users: User)

    @Delete
    suspend fun delete(user: User)

    @Query("SELECT * FROM user")
    suspend fun getAll(): List<User>
}
```



Room



Criando o projeto Exemplo

New Project

Empty Activity

Create a new empty activity with Jetpack Compose

Name RoomSQLite

Package name leite.sampaio.lucas.roomsqlite

Save location /home/lucas/AndroidStudioProjects/RoomSQLite

Minimum SDK API 24 ("Nougat"; Android 7.0)

i Your app will run on approximately 99,2% of devices.
[Help me choose](#)

Build configuration language ? Kotlin DSL (build.gradle.kts) [Recommended]

Previous Next Cancel Finish

Configurando dependências manualmente

- gradle/libs.versions.toml

```
[versions]
sqliteBundled = "2.7.1"
recyclerview = "1.4.0"
room3Compile = "3.0.3"
room3Runtime = "3.0.3"
ksp = "2.3.10"
```

```
[libraries]
androidx-sqlite-bundled = { group = "androidx.sqlite", name = "sqlite-bundled", version.ref = "sqliteBundled" }
androidx-recyclerview = { group = "androidx.recyclerview", name = "recyclerview", version.ref = "recyclerview" }
androidx-room3-compiler = { module = "androidx.room3:room3-compiler", version.ref = "room3Compile" }
androidx-room3-runtime = { module = "androidx.room3:room3-runtime", version.ref = "room3Runtime" }
```

```
[plugins]
google-devtools-ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Configurando dependências manualmente

- gradle/libs.versions.toml

O `libs.versions.toml` é um arquivo usado pelo Gradle Version Catalog para centralizar as versões e as declarações das dependências de um projeto Android.

```
[versions]
sqliteBundled = "2.7.1"
recyclerview = "1.4.0"
room3Compile = "3.0.3"
room3Runtime = "3.0.3"
ksp = "2.3.10"
```

```
[libraries]
androidx-sqlite-bundled = { group = "androidx.sqlite", name = "sqlite-bundled", version.ref = "sqliteBundled" }
androidx-recyclerview = { group = "androidx.recyclerview", name = "recyclerview", version.ref = "recyclerview" }
androidx-room3-compiler = { module = "androidx.room3:room3-compiler", version.ref = "room3Compile" }
androidx-room3-runtime = { module = "androidx.room3:room3-runtime", version.ref = "room3Runtime" }
```

```
[plugins]
google-devtools-ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

Configurando dependências manualmente

- build.gradle.kts (Project:RoomSQLite)

```
plugins {  
    alias(libs.plugins.google.devtools.ksp) apply false
```

- build.gradle.kts (Module:app)

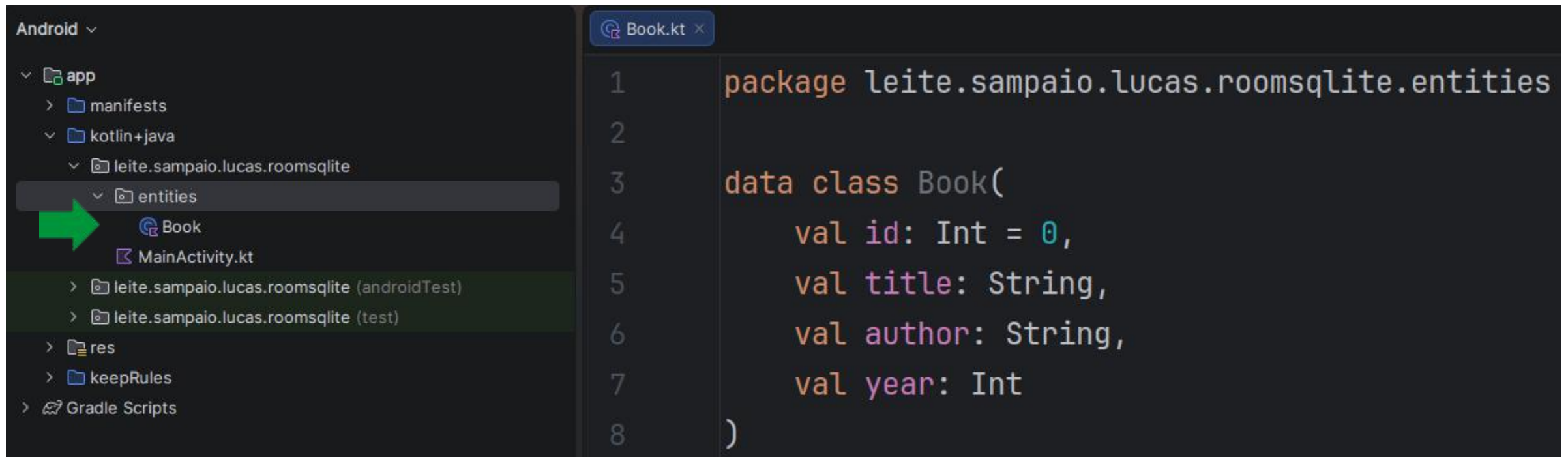
```
plugins {  
    alias(libs.plugins.google.devtools.ksp)
```

```
dependencies {  
    implementation(libs.androidx.recyclerview)  
    implementation(libs.androidx.sqlite.bundled)  
    implementation(libs.androidx.room3.runtime)  
    ksp(libs.androidx.room3.compiler)
```

Dependências x Plugins

- Dependência: Uma dependência é uma biblioteca que o aplicativo utiliza para adicionar funcionalidades ao código da aplicação.
- Plugin: Um plugin adiciona funcionalidades ao próprio sistema de construção do Gradle.
 - Por exemplo: `alias(libs.plugins.android.application)`
 - Esse plugin informa ao Gradle como construir um aplicativo Android.
 - O KSP permite que ferramentas como o Room realizem processamento de código durante a compilação.

Criando a data class Book



```
1 package leite.sampaio.lucas.roomsqlite.entities
2
3 data class Book(
4     val id: Int = 0,
5     val title: String,
6     val author: String,
7     val year: Int
8 )
```

Transformando a data class Book em Entity

```
import androidx.room3.Entity
import androidx.room3.PrimaryKey

36 Usages
@Entity(tableName = "books")
data class Book(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val title: String,
    val author: String,
    val year: Int
)
```

Transformando a data class Book em Entity

```
import androidx.room3.Entity
import androidx.room3.PrimaryKey

36 Usages

@Entity(tableName = "books")
data class Book(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val title: String,
    val author: String,
    val year: Int
)
```



Criando BookDao

```
import androidx.room3.Dao
import androidx.room3.Delete
import androidx.room3.Insert
import androidx.room3.Query
import androidx.room3.Update
import kotlinx.coroutines.flow.Flow
```

```
@Dao
interface BookDao {
```

1 Usage 1 Implementation

```
@Insert
```

```
suspend fun insert(book: Book)
```

1 Usage 1 Implementation

```
@Update
```

```
suspend fun update(book: Book)
```

1 Usage 1 Implementation

```
@Delete
```

```
suspend fun delete(book: Book)
```

1 Usage 1 Implementation

```
@Query(value = "SELECT * FROM books ORDER BY title")
```

```
fun listAll(): Flow<List<Book>>
```

1 Implementation

```
@Query(value = "SELECT * FROM books WHERE id = :id")
```

```
suspend fun findById(id: Int): Book?
```

```
}
```

Criando BookDao

```
import androidx.room3.Dao
import androidx.room3.Delete
import androidx.room3.Insert
import androidx.room3.Query
import androidx.room3.Update
import kotlinx.coroutines.flow.Flow
```

Permite manter a lista da interface sincronizada com o banco. Ou seja, emite uma nova lista sempre que os dados observador forem alterados.

```
@Dao
interface BookDao {
```

1 Usage 1 Implementation

@Insert

suspend fun insert(book: Book)

1 Usage 1 Implementation

@Update

suspend fun update(book: Book)

1 Usage 1 Implementation

@Delete

suspend fun delete(book: Book)

1 Usage 1 Implementation

@Query(value = "SELECT * FROM books ORDER BY title")

fun listAll(): Flow<List<Book>>

1 Implementation

@Query(value = "SELECT * FROM books WHERE id = :id")

suspend fun findById(id: Int): Book?

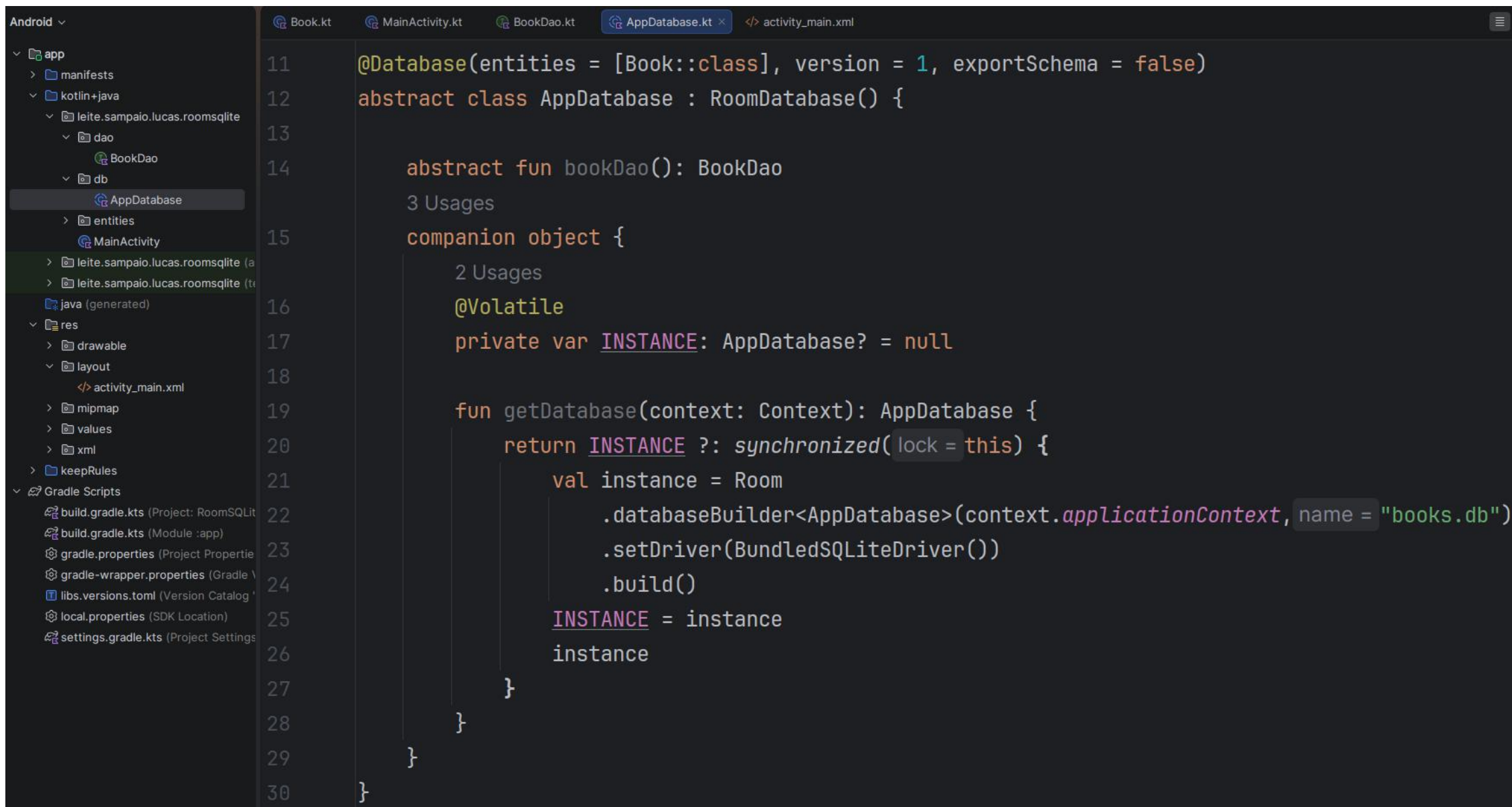
}

Room x SQLite nativo

```
override fun onCreate(db: SQLiteDatabase) {  
    val sql = String.format(  
        "CREATE TABLE %s (%s integer primary key autoincrement, %s text, %s text, %s text)",  
        TABLE, ID, TITLE, AUTHOR, PUBLISHER  
    )  
    db.execSQL(sql)  
}
```

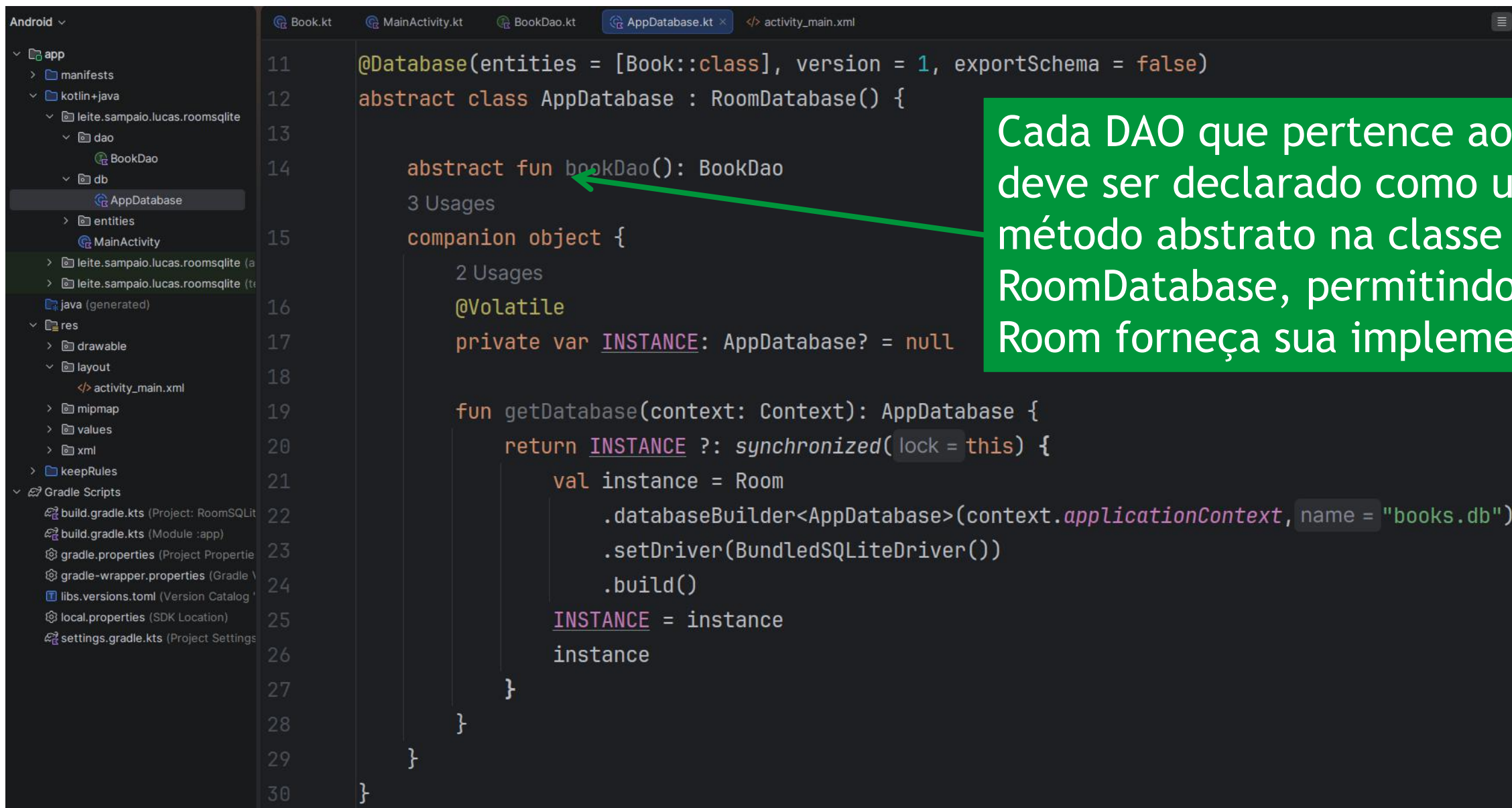
```
fun insertData(title: String, author: String, publisher: String): String {  
    val values = ContentValues()  
    values.put(CreateDB.TITLE, title)  
    values.put(CreateDB.AUTHOR, author)  
    values.put(CreateDB.PUBLISHER, publisher)  
    val result = writableDatabase.insert(CreateDB.TABLE, nullColumnHack: null, values)  
    return if (result == -1L) "Error inserting record" else "Record inserted successfully"  
}
```


Criando o Banco de Dados



```
11 @Database(entities = [Book::class], version = 1, exportSchema = false)
12 abstract class AppDatabase : RoomDatabase() {
13
14     abstract fun bookDao(): BookDao
15     3 Usages
16     companion object {
17         2 Usages
18         @Volatile
19         private var INSTANCE: AppDatabase? = null
20
21         fun getDatabase(context: Context): AppDatabase {
22             return INSTANCE ?: synchronized(lock = this) {
23                 val instance = Room
24                 .databaseBuilder<AppDatabase>(context.applicationContext, name = "books.db")
25                 .setDriver(BundledSQLiteDriver())
26                 .build()
27                 INSTANCE = instance
28                 instance
29             }
30         }
31     }
32 }
```

Criando o Banco de Dados



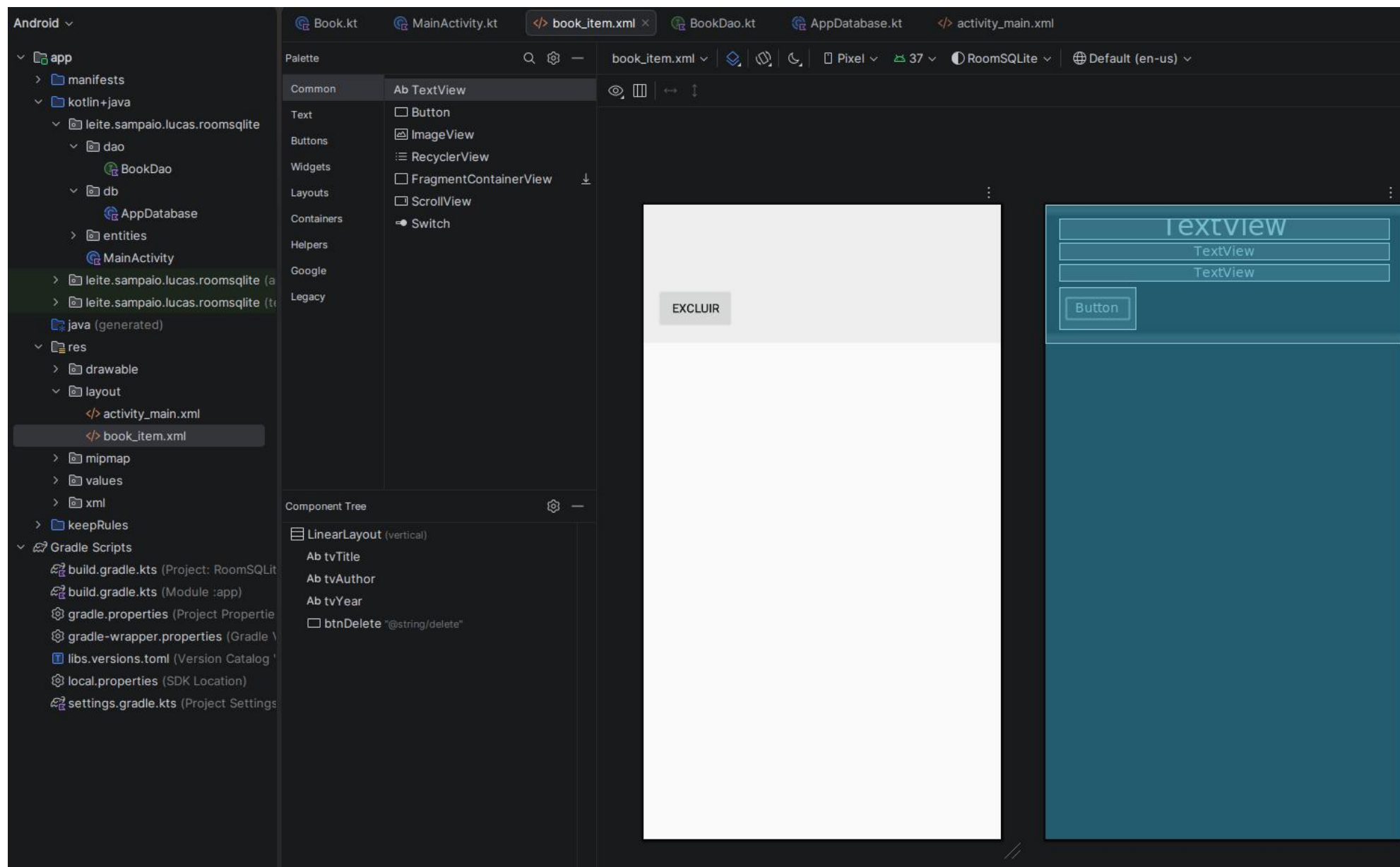
```
11 @Database(entities = [Book::class], version = 1, exportSchema = false)
12 abstract class AppDatabase : RoomDatabase() {
13
14     abstract fun bookDao(): BookDao
15     3 Usages
16     companion object {
17         2 Usages
18         @Volatile
19         private var INSTANCE: AppDatabase? = null
20
21         fun getInstance(context: Context): AppDatabase {
22             return INSTANCE ?: synchronized(lock = this) {
23                 val instance = Room
24                 .databaseBuilder<AppDatabase>(context.applicationContext, name = "books.db")
25                 .setDriver(BundledSQLiteDriver())
26                 .build()
27                 INSTANCE = instance
28                 instance
29             }
30     }
```

Cada DAO que pertence ao banco deve ser declarado como um método abstrato na classe RoomDatabase, permitindo que o Room forneça sua implementação.

Criando o Layout da Activity

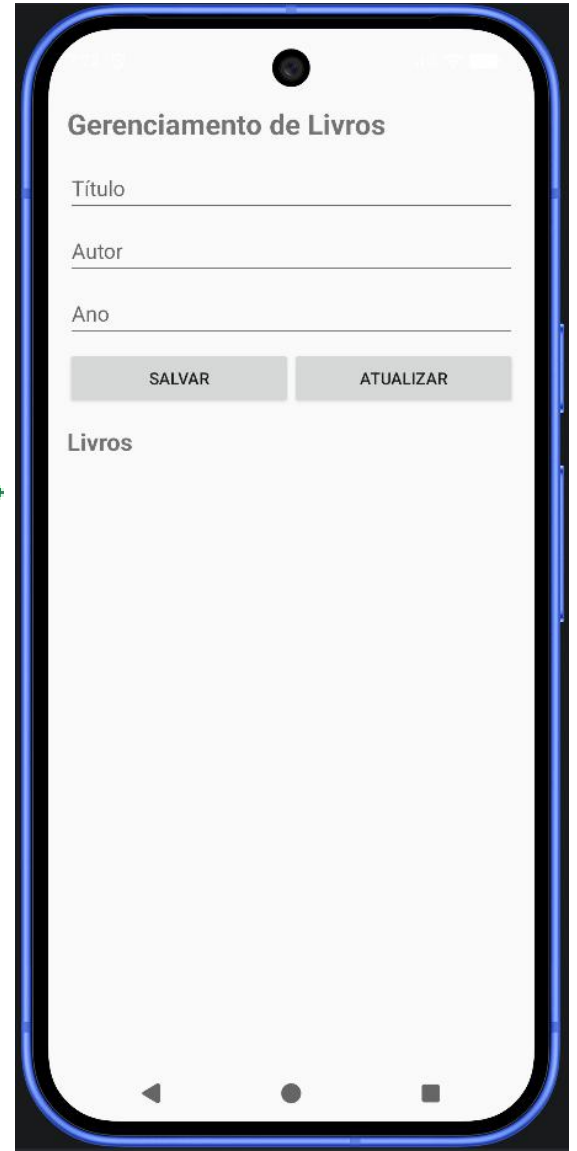
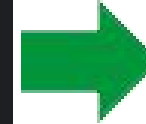


Criando o Layout do Item



Executando a aplicação

```
class MainActivity : ComponentActivity() {  
  
    2 Usages  
    private lateinit var binding: ActivityMainBinding  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        binding = ActivityMainBinding.inflate(layoutInflater)  
        setContentView(binding.root)  
    }  
}
```



Gerenciamento de Livros

Título

Autor

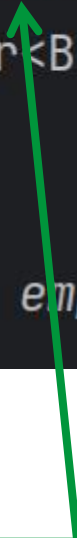
Ano

SALVAR ATUALIZAR

Livros

Implementando o adapter

```
class BookAdapter(  
    private val onItemClick: (Book) -> Unit,  
    private val onDeleteClick: (Book) -> Unit  
) : RecyclerView.Adapter<BookAdapter.BookViewHolder>() {  
  
    3 Usages  
    private var books = emptyList<Book>()  
}
```



O Adapter recebe funções de callback para comunicar à Activity quais interações ocorreram, permitindo que a Activity defina o comportamento de cada evento.

Implementando o adapter

```
class BookViewHolder(private val binding: BookItemBinding) :  
    RecyclerView.ViewHolder(itemView = binding.root) {  
  
    1 Usage  
  
    fun bind(book: Book, onItemClick: (Book) -> Unit, onDeleteClick: (Book) -> Unit) {  
        binding.tvTitle.text = book.title  
        binding.tvAuthor.text = "Autor: ${book.author}"  
        binding.tvYear.text = "Ano: ${book.year}"  
  
        binding.root.setOnClickListener {  
            onItemClick(book)  
        }  
  
        binding.btnDelete.setOnClickListener {  
            onDeleteClick(book)  
        }  
    }  
}
```

A função `bind()` é responsável por receber um objeto `Book` e preencher um item do `RecyclerView` com os seus dados, além de configurar os eventos de clique.

Implementando o adapter

```
override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): BookViewHolder {  
    val binding = BookItemBinding.inflate(inflater = LayoutInflater.from(parent.context),  
        parent, attachToParent = false)  
    return BookViewHolder(binding)  
}  
  
override fun onBindViewHolder(holder: BookViewHolder, position: Int) {  
    holder.bind(books[position], onItemClick, onDeleteClick)  
}  
  
override fun getItemCount(): Int {  
    return books.size  
}  
  
fun updateList(novosLivros: List<Book>) {  
    books = novosLivros  
    notifyDataSetChanged()  
}
```

A função `notifyDataSetChanged()` informa ao RecyclerView que os dados utilizados pelo Adapter foram alterados e que a lista exibida precisa ser atualizada.

Implementando a Activity

```
private lateinit var database: AppDatabase
1 Usage
private lateinit var bookDao: BookDao
private lateinit var adapter: BookAdapter

private var selectedBook: Book? = null

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)

    database = AppDatabase.getDatabase(context = this)
    bookDao = database.bookDao()

    configureRecyclerView()
    configureButtons()
    observeBooks()
}
```


Implementando a Activity

```
private fun configureRecyclerView() {  
    adapter = BookAdapter(  
        onItemClick = { book -> selectBook(book) },  
        onDeleteClick = { book -> deleteBook(book) }  
    )  
    binding.rvBooks.layoutManager = LinearLayoutManager(context = this)  
  
    binding.rvBooks.adapter = adapter  
}
```


Implementando a Activity

```
private fun configureButtons() {  
    binding.btnSave.setOnClickListener {  
        insertBook()  
    }  
  
    binding.btnUpdate.setOnClickListener {  
        updateBook()  
    }  
}
```

Implementando a Activity

```
private fun insertBook() {  
    val title = binding.etTitle.text.toString()  
    val author = binding.etAuthor.text.toString()  
    val year = binding.etYear.text.toString()  
  
    if (title.isBlank() || author.isBlank() || year.isBlank()) {  
        Toast.makeText(context = this, text = "Preencha todos os campos.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
  
    val book = Book(title = title, author = author, year = year.toInt())  
  
    lifecycleScope.launch {  
        bookDao.insert(book)  
        clearForm()  
        Toast.makeText(context = this@MainActivity, text = "Livro inserido.", duration = Toast.LENGTH_SHORT).show()  
    }  
}
```

Implementando a Activity

```
private fun insertBook() {  
    val title = binding.etTitle.text.toString()  
    val author = binding.etAuthor.text.toString()  
    val year = binding.etYear.text.toString()  
  
    if (title.isBlank() || author.isBlank() || year.isBlank()) {  
        Toast.makeText(context = this, text = "Preencha todos os campos.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
  
    val book = Book(title = title, author = author, year = year)  
  
    lifecycleScope.launch {  
        bookDao.insert(book)  
        clearForm()  
        Toast.makeText(context = this@MainActivity, text = "Livro inserido.", duration = Toast.LENGTH_SHORT).show()  
    }  
}
```

`lifecycleScope.launch` inicia uma coroutine vinculada ao ciclo de vida da Activity, permitindo executar funções suspend, como operações do Room, sem bloquear a interface do aplicativo.

Implementando a Activity

```
private fun clearForm() {  
    binding.etTitle.text.clear()  
    binding.etAuthor.text.clear()  
    binding.etYear.text.clear()  
  
    selectedBook = null  
}
```

Implementando a Activity

```
private fun updateBook() {  
    val book = selectedBook  
  
    if (book == null) {  
        Toast.makeText(context = this, text = "Selecione um livro.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
  
    val title = binding.etTitle.text.toString()  
    val author = binding.etAuthor.text.toString()  
    val year = binding.etYear.text.toString()  
  
    if (title.isBlank() || author.isBlank() || year.isBlank() ) {  
        Toast.makeText(context = this, text = "Preencha todos os campos.", duration = Toast.LENGTH_SHORT).show()  
        return  
    }  
}
```



Implementando a Activity



```
val updatedBook = book.copy(title = title, author = author, year = year.toInt())

lifecycleScope.launch {
    bookDao.update(livro = updatedBook)
    selectedBook = null
    clearForm()
    Toast.makeText(context = this@MainActivity, text = "Livro atualizado.", duration = Toast.LENGTH_SHORT).show()
}
```


Implementando a Activity

```
private fun deleteBook(book: Book) {  
    lifecycleScope.launch {  
        bookDao.delete(book)  
  
        if (selectedBook?.id == book.id) {  
            selectedBook = null  
            clearForm()  
        }  
  
        Toast.makeText(context = this@MainActivity, text = "Livro excluído.", duration = Toast.LENGTH_SHORT).show()  
    }  
}
```

Implementando a Activity

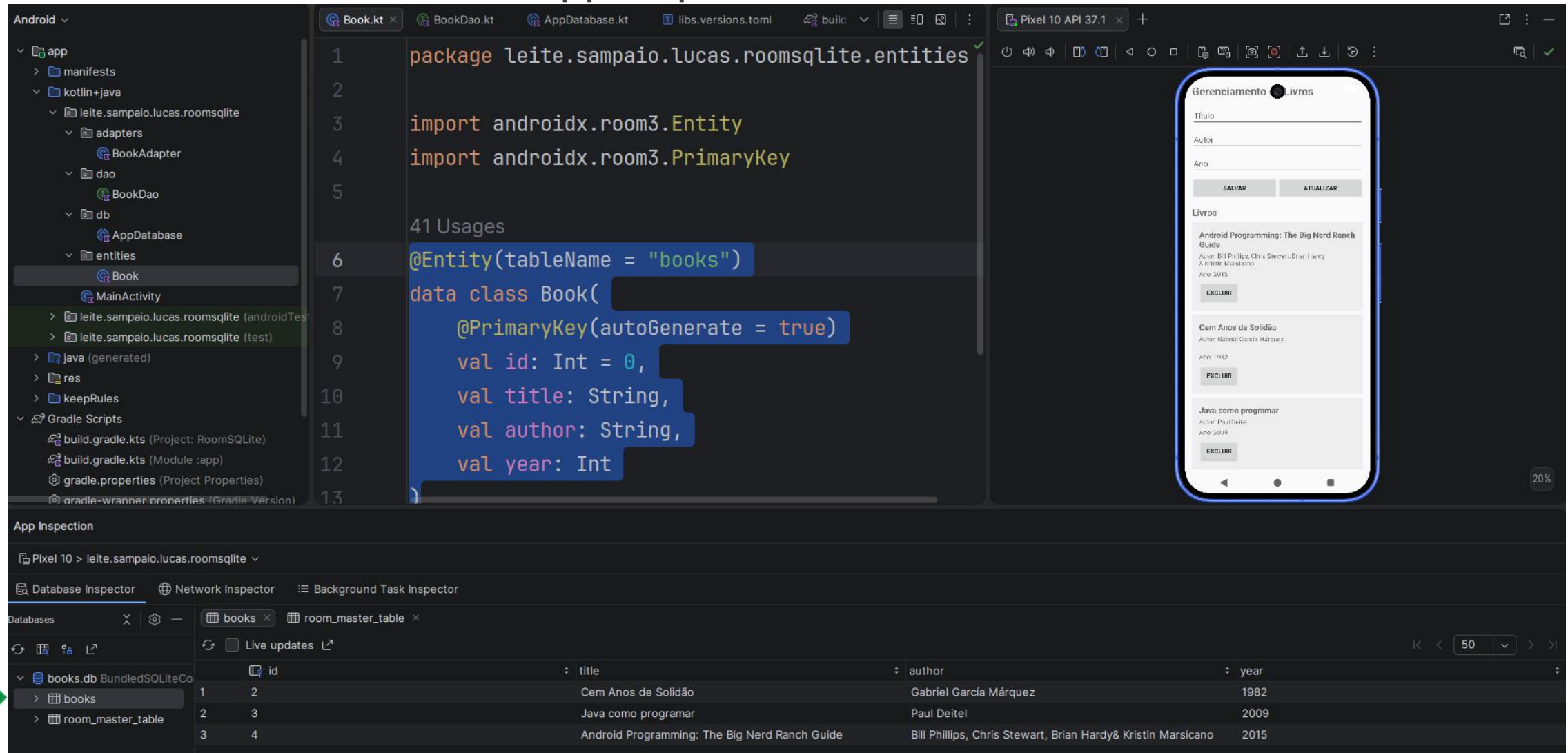
```
private fun selectBook(book: Book) {  
    selectedBook = book  
  
    binding.etTitle.setText(book.title)  
    binding.etAuthor.setText(book.author)  
    binding.etYear.setText(book.year.toString())  
  
    Toast.makeText(context = this, text = "Livro selecionado.", duration = Toast.LENGTH_SHORT).show()  
}
```

Implementando a Activity

```
private fun observeBooks() {  
    lifecycleScope.launch {  
        bookDao.listAll().collect { books -> adapter.updateList(novosLivros = books) }  
    }  
}
```

Visualizando books.db

- View -> Tool Windows -> App Inspector



The screenshot displays the Android Studio interface. The top editor shows the `Book.kt` file with the following code:

```
1 package leite.sampaio.lucas.roomsqlite.entities
2
3 import androidx.room3.Entity
4 import androidx.room3.PrimaryKey
5
6 @Entity(tableName = "books")
7 data class Book(
8     @PrimaryKey(autoGenerate = true)
9     val id: Int = 0,
10    val title: String,
11    val author: String,
12    val year: Int
13 )
```

The bottom part of the screen shows the App Inspector tool window. It displays the database structure for `books.db` and a table of books.

id	title	author	year
1	Cem Anos de Solidão	Gabriel García Márquez	1982
2	Java como programar	Paul Deitel	2009
3	Android Programming: The Big Nerd Ranch Guide	Bill Phillips, Chris Stewart, Brian Hardy & Kristin Marsicano	2015

Exercícios

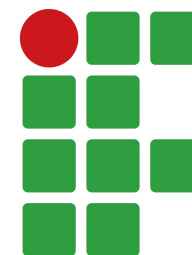
- Considere o aplicativo de gerenciamento de livros desenvolvido em aula e realize as modificações solicitadas abaixo:
 - Identifique todas as strings que estão diretamente escritas nos arquivos Kotlin e XML do aplicativo e mova-as para o arquivo `res/values/strings.xml`.
 - Modifique o aplicativo para permitir o cadastro e a exibição da editora do livro.
 - Atualmente, o método `updateList()` do `BookAdapter` está implementado utilizando `notifyDataSetChanged()`, que informa ao `RecyclerView` que houve alterações na lista, mas não identifica quais itens foram alterados. Modifique o `BookAdapter` para utilizar `DiffUtil` no lugar de `notifyDataSetChanged()`.
 - <https://developer.android.com/reference/androidx/recyclerview/widget/DiffUtil>
 - <https://www.geeksforgeeks.org/android/diffutil-in-recyclerview-in-android/>

Dúvidas



PROGRAMAÇÃO PARA DISPOSITIVOS MÓVEIS

Curso de Engenharia de Software
Lucas Sampaio Leite



**INSTITUTO
FEDERAL**
Pernambuco